

Week 2

Non-linear separable Boolean Functions

Representation Power of a Network of Perceptrons

Sigmoid Neuron

Perceptron vs Sigmoid

A Typical Supervised Machine Learning Setup

Data

Model

Parameters

Learning Algorithm

Objective/Loss/Error Function

Learning Parameters

Guess Work (Infeasible)

Input for Training

Training Objective

Gradient Descent

Taylor Series

Gradient Descent Algorithm

Algorithm

Code

Representation Power of a Multilayer Network of Sigmoid Neurons

Illustration

Non-linear separable Boolean Functions

x_1	x_2	XOR	
0	0	0	$w_0 + \sum_{i=1}^2 w_i x_i < 0$
1	0	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$
0	1	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$
1	1	0	$w_0 + \sum_{i=1}^2 w_i x_i < 0$

$$w_0 + w_1 \cdot 0 + w_2 \cdot 0 < 0 \implies w_0 < 0$$

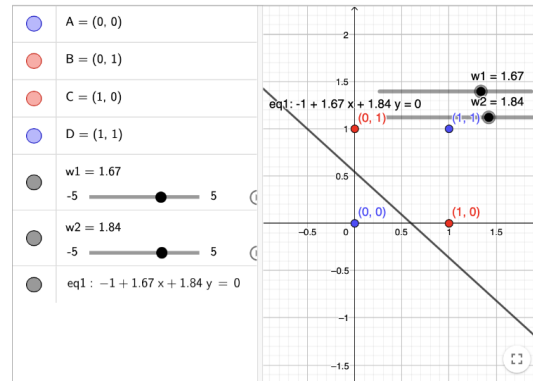
$$w_0 + w_1 \cdot 0 + w_2 \cdot 1 \geq 0 \implies w_2 \geq -w_0$$

$$w_0 + w_1 \cdot 1 + w_2 \cdot 0 \geq 0 \implies w_1 \geq -w_0$$

$$w_0 + w_1 \cdot 1 + w_2 \cdot 1 < 0 \implies w_1 + w_2 < -w_0$$

The fourth condition contradicts conditions 2 and 3

Hence we cannot have a solution to this set of inequalities

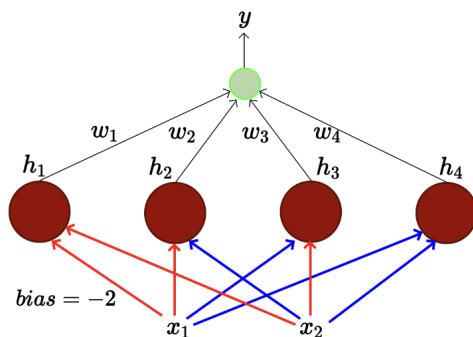


And indeed you can see that it is impossible to draw a line which separates the red points from the blue points



There are 2^{2^n} boolean functions for n inputs.

Representation Power of a Network of Perceptrons



red edge indicates $w = -1$

blue edge indicates $w = +1$

Terminology:

This network contains 3 layers

The layer containing the inputs (x_1, x_2) is called the **input layer**

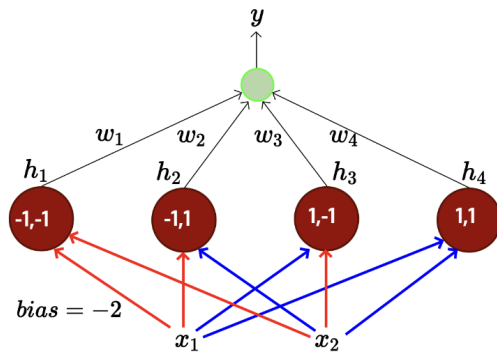
The middle layer containing the 4 perceptrons is called the **hidden layer**

The final layer containing one output neuron is called the **output layer**

The outputs of the 4 perceptrons in the hidden layer are denoted by h_1, h_2, h_3, h_4

The red and blue edges are called layer 1 weights

w_1, w_2, w_3, w_4 are called layer 2 weights



red edge indicates $w = -1$
blue edge indicates $w = +1$

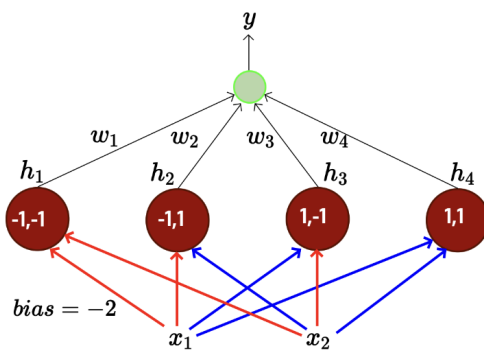
We claim that this network can be used to implement **any** boolean function (linearly separable or not)!

In other words, we can find w_1, w_2, w_3, w_4 such that the truth table of any boolean function can be represented by this network

Astonishing claim! Well, not really, if you understand what is going on

Each perceptron in the middle layer fires only for a specific input (and no two perceptrons fire for the same input)

the fourth perceptron fires for $\{1,1\}$



red edge indicates $w = -1$
blue edge indicates $w = +1$

Let w_0 be the bias output of the neuron (i.e., it will fire if $\sum_{i=1}^4 w_i h_i \geq w_0$)

x_1	x_2	XOR	h_1	h_2	h_3	h_4	$\sum_{i=1}^4 w_i h_i$
0	0	0	1	0	0	0	w_1
0	1	1	0	1	0	0	w_2
1	0	1	0	0	1	0	w_3
1	1	0	0	0	1	1	w_4

This results in the following four conditions to implement XOR: $w_1 < w_0, w_2 \geq w_0, w_3 \geq w_0, w_4 < w_0$

Unlike before, there are no contradictions now and the system of inequalities can be satisfied

Essentially each w_i is now responsible for one of the 4 possible inputs and can be adjusted to get the desired output for that input



Any boolean function of n inputs can be represented exactly by a network of perceptrons containing 1 hidden layer with 2^n perceptrons and one output layer containing 1 perceptron.

Note: A network of $2^n + 1$ perceptrons is not necessary but sufficient

Sigmoid Neuron



A sigmoid fires only if the weighted sum of its inputs is greater than the threshold $(-w_0)$

Let $y = f(x)$

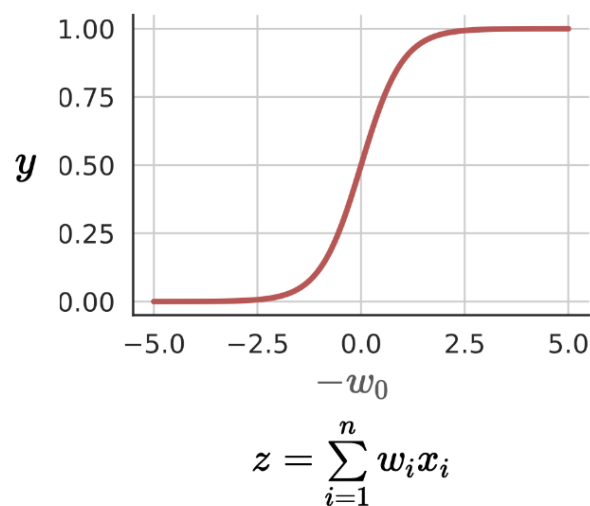
where, $x \in \mathbb{R}^n$ and

$y \in \mathbb{R}$

- The thresholding logic used by a perceptron is very harsh. It is a characteristic of the perceptron function itself which behaves like a step function.
- For most real world applications we would expect a smoother decision function which gradually changes from 0 to 1.
- **Sigmoid Function:**

$$y = \frac{1}{1 + e^{-(w_0 + \sum_{i=1}^n w_i x_i)}}$$

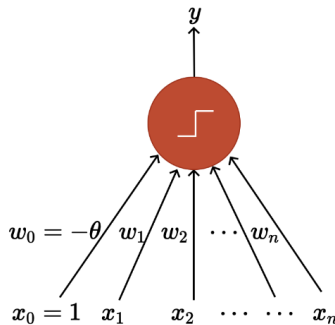
- We no longer see a sharp transition around the threshold w_0 .



- Output y is a real value between 0 and 1 which can be interpreted as a probability.

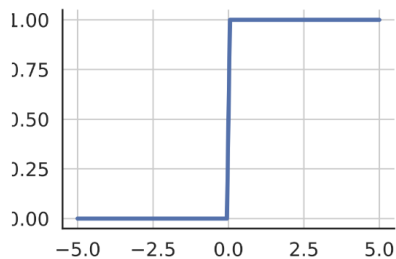
Perceptron vs Sigmoid

Perceptron



$$y = 1 \text{ if } \sum_{i=0}^n w_i x_i \geq 0$$

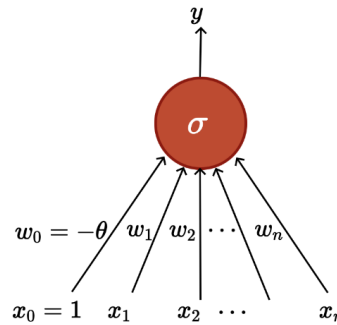
$$= 0 \text{ if } \sum_{i=0}^n w_i x_i < 0$$



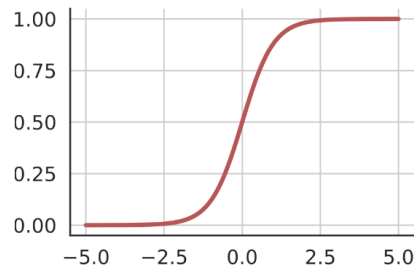
$$z = \sum_{i=1}^n w_i x_i - w_0$$

Not smooth,
not differentiable at $(-w_0)$

Sigmoid (Logistic) Neuron



$$y = \frac{1}{1 + \exp\left(-\left(w_0 + \sum_{i=1}^n w_i x_i\right)\right)}$$



$$z = \sum_{i=1}^n w_i x_i - w_0$$

Smooth, continuous,
differentiable

A Typical Supervised Machine Learning Setup

Data

$$\{x_i, y_i\}_{i=1}^n$$

Model

Our approximation of the relation between x and y .

For example, $\hat{y} = \frac{1}{1+e^{w^T x}}$ or $\hat{y} = w^T x$ or $\hat{y} = x^T W x$

Parameters

In all the above cases, w is a parameter which needs to be learned from the data.

Learning Algorithm

An algorithm for learning the parameters w of the model (for example, perceptron learning algorithm, gradient descent, etc.)

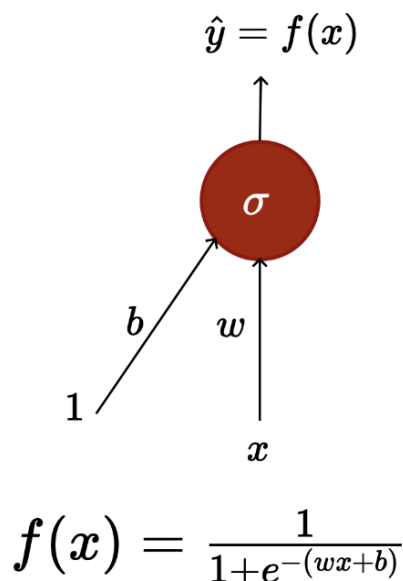
Objective/Loss/Error Function

To guide the learning algorithm

the learning algorithm should aim to minimize the loss function

Learning Parameters

Guess Work (Infeasible)



Input for Training

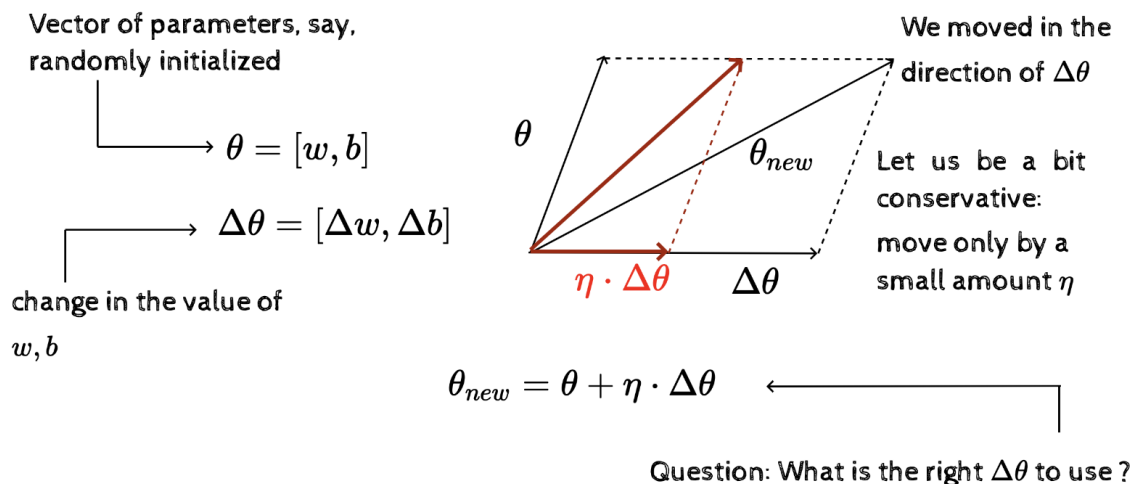
$$\{x_i, y_i\}_{i=1}^N \longrightarrow \text{N pairs of } (x, y)$$

Training Objective

Find w and b such that

$$\min_{(w,b)} \in \mathbb{R} L(w, b) = \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))^2$$

Gradient Descent



The answer comes from **Taylor series**

Taylor Series

Taylor series is a way of approximating any continuously differentiable function $L()$ using polynomials of degree n . The higher the degree the better the approximation.

$$L(w) = L(w_0) + \frac{L'(w_0)}{1!}(w - w_0) + \frac{L'(w_0)}{2!}(w - w_0)^2 + \frac{L'(w_0)}{3!}(w - w_0)^3 + \dots$$

Gradient Descent Algorithm

For ease of notation, let $\Delta\theta = u$, then from Taylor series, we have,

$$\begin{aligned}\mathcal{L}(\theta + \eta u) &= \mathcal{L}(\theta) + \eta * u^T \nabla_{\theta} \mathcal{L}(\theta) + \frac{\eta^2}{2!} * u^T \nabla_{\theta}^2 \mathcal{L}(\theta) u + \frac{\eta^3}{3!} \dots + \frac{\eta^4}{4!} \dots \\ &= \mathcal{L}(\theta) + \eta * u^T \nabla_{\theta} \mathcal{L}(\theta) \quad [\eta \text{ is typically small, so } \eta^2, \eta^3 \dots \rightarrow 0]\end{aligned}$$

Note that the move ηu would be favorable only if

$$\mathcal{L}(\theta + \eta u) - \mathcal{L}(\theta) < 0 \quad [\text{i.e., if the new loss is less than the previous loss}]$$

This implies,

$$u^T \nabla_{\theta} \mathcal{L}(\theta) < 0$$

Let β be the angle between u^T and $\nabla_{\theta} \mathcal{L}(\theta)$, then we know that

$$-1 \leq \cos(\beta) = \frac{u^T \nabla_{\theta} \mathcal{L}(\theta)}{\|u^T\| * \|\nabla_{\theta} \mathcal{L}(\theta)\|} \leq 1$$

Multiply through $k = \|u^T\| * \|\nabla_{\theta} \mathcal{L}(\theta)\|$

$$-k \leq k * \cos(\beta) = u^T \nabla_{\theta} \mathcal{L}(\theta) \leq k$$

Thus $\mathcal{L}(\theta + \eta u) - \mathcal{L}(\theta) = u^T \nabla_{\theta} \mathcal{L}(\theta) = k * \cos(\beta)$ is most negative when $\cos(\beta) = -1$ (i.e., $\beta = 180^\circ$)



Gradient Descent Rule

The direction u that we intend to move in should be at 180° w.r.t. the gradient. In other words, move in a direction opposite to the gradient



Parameter Updation Rule

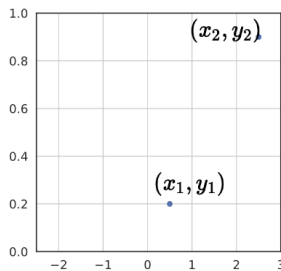
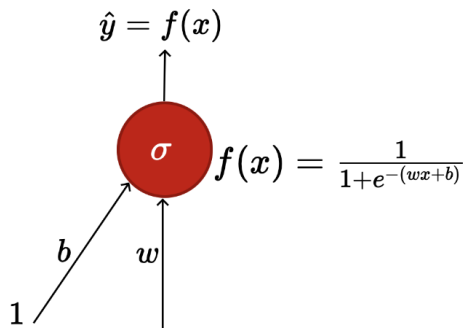
$$\begin{aligned}w_{t+1} &= w_t - \eta \nabla w_t \\b_{t+1} &= b_t - \eta \nabla b_t\end{aligned}$$

where, $\nabla w_t = \frac{\partial L(w, b)}{\partial w}$, at $w = w_t$ and $b = b_t$

where, $\nabla b_t = \frac{\partial L(w, b)}{\partial b}$, at $w = w_t$ and $b = b_t$

Algorithm

```
t ← 0;
max_iterarions ← 1000;
w,b ← initialize randomly
while t < max_iterarions do
    |  $w_{t+1} \leftarrow w_t - \eta \nabla w_t;$ 
    |  $b_{t+1} \leftarrow b_t - \eta \nabla b_t;$ 
    |  $t \leftarrow t + 1;$ 
end
```



So If we have only one point (x, y) , we have,

$$\nabla w = (f(x) - y) * f(x) * (1 - f(x)) * x$$

For two points,

$$\nabla w = \sum_{i=1}^2 (f(x_i) - y_i) * f(x_i) * (1 - f(x_i)) * x_i$$

$$\nabla b = \sum_{i=1}^2 (f(x_i) - y_i) * f(x_i) * (1 - f(x_i))$$

Code

```
import numpy as np
X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(x,w,b):
    return 1/(1+np.exp(-(w*x+b)))

def error(w,b):
    err = 0.0
    for x,y in zip(X,Y):
        fx = f(x,w,b)
        err += (fx-y)**2
    return 0.5*err

def grad_b(x,w,b,y):
    fx = f(x,w,b)
    return (fx-y)*fx*(1-fx)

def grad_w(x,w,b,y):
    fx = f(x,w,b)
    return (fx-y)*fx*(1-fx)*x

def do_gradient_descent():
    w,b,eta,max_epochs = -2, -2, 1.0, 1000

    for i in range(max_epochs):
        dw,db = 0,0
        for x,y in zip(X,Y):
```

```
dw += grad_w(x,w,b,y)
db += grad_b(x,w,b,y)
```

```
w = w - eta*dw
b = b - eta*db
```

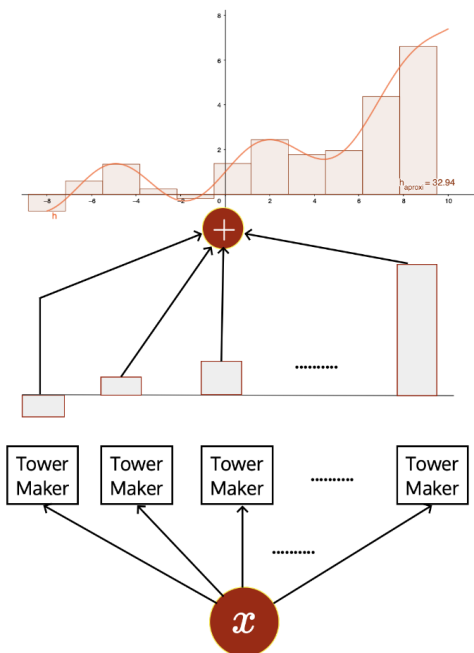
Representation Power of a Multilayer Network of Sigmoid Neurons



A multilayer network of neurons with a single hidden layer can be used to approximate any continuous function to any desired precision.

In other words, there is a **guarantee** that for any function $f(x) : \mathbb{R}^n \rightarrow \mathbb{R}^m$, we can always find a neural network (with 1 hidden layer containing enough neurons) whose output $g(x)$ satisfies $|g(x) - f(x)| < \epsilon$

Illustration



Any function can be **represented** by summing **shifted** and **scaled** versions of tower functions

Suppose there is a black box which takes the original input (x) and constructs these tower functions

We can then have a simple network which can just add them up to approximate the function

Our job now is to figure out what is inside this blackbox

